

Table of Contents

1. Project Overview	2
2. Technology Stack	2
2.1 Core Frameworks and Libraries	2
2.2 Development Tools	4
3. System Architecture	4
3.1 High-Level Architecture	4
3.2 Frontend Architecture	4
3.3 Backend Architecture	5
3.4 Authentication and Authorization	5
4. Database Schema	5
4.1 Core Domain Models	6
4.2 Approval Workflow Models	6
4.3 Supporting Models	7
5. API Routes	7
6. Key Features	9
6.1 Document Management	9
6.2 Approval Workflows	10
6.3 Dashboard and Analytics	10
6.4 Additional Features	11
7. Project Structure	11
8. Library Modules	12
9. Deployment	13
10. Document Lifecycle	14

1. Project Overview

DocsFlowBPM (version 0.2.0) is a full-stack BPM (Business Process Management) system designed for document workflow automation and approval routing. The application provides a comprehensive document management solution that supports document creation with custom form schemas, folder-based organization with drag-and-drop, multi-step approval workflows with SLA tracking, role-based access control, and real-time notifications. The system is built as a single-page application (SPA) with a rich admin panel, analytics dashboard, command palette, and both light and dark theme support. All interfaces are localized in Russian, targeting enterprise document management scenarios in Russian-speaking organizations.

The core concept revolves around documents that move through defined lifecycle states (Draft, In Progress, Approved, Rejected, Completed) and can be routed through configurable approval chains. Each approval step can be assigned to a specific user or an entire department, with optional SLA deadlines calculated based on document urgency and configurable working hours. The system features a powerful formula evaluator that allows computed fields in document forms, a versatile permission model granting VIEW and EDIT access per document, and browser-based desktop notifications for time-sensitive approval events.

2. Technology Stack

2.1 Core Frameworks and Libraries

The application is built on modern web technologies, combining a powerful React-based frontend with server-side rendering capabilities. Below is a comprehensive overview of all major technologies used in the project, organized by their functional role within the system.

Technology	Version	Category	Purpose
Next.js	16.1.1	Framework	Full-stack React framework with App Router, API routes, and standalone output mode
React	19.0.0	UI Library	Component-based UI rendering with hooks and concurrent features
TypeScript	5.x	Language	Static type checking for improved code quality and IDE support
Prisma	6.11.1	ORM	Type-safe database access with schema-first approach and migrations
PostgreSQL	-	Database	Relational database for all persistent data storage

Tailwind CSS	4.x	Styling	Utility-first CSS framework with dark mode class strategy
shadcn/ui	-	UI Kit	35+ accessible UI components built on Radix UI primitives
Radix UI	Various	Primitives	Accessible headless UI components for dialogs, dropdowns, etc.
Zustand	5.0.6	State Mgmt	Lightweight client-side state management for auth, navigation, and data cache
TanStack React Query	5.82.0	Data Fetching	Server state management with caching and background refetch
TanStack React Table	8.21.3	Table	Headless table utility for advanced data grid features
Recharts	2.15.4	Charts	Declarative charting library for pie and bar charts in analytics
Framer Motion	12.2.3.2	Animation	Production-ready animation library for smooth UI transitions
@dnd-kit	6.3.1+	DnD	Accessible drag-and-drop for document-to-folder operations
React Hook Form	7.60.0	Forms	Performant form state management with validation
Zod	4.0.2	Validation	TypeScript-first schema validation for form inputs
next-themes	0.4.6	Theming	Dynamic theme switching with system preference detection
Sonner	2.0.6	Notifications	Toast notification system with rich formatting
Lucide React	0.525.0	Icons	Open-source icon library with 500+ icons
cmdk	1.1.1	Command Palette	Command menu component for Ctrl+K navigation
bcryptjs	3.0.3	Security	Password hashing with 12 salt rounds for authentication
Sharp	0.34.3	Images	High-performance image processing library
Caddy	-	Reverse Proxy	Production reverse proxy with port transformation support
Bun	-	Runtime	JavaScript runtime used for production server execution
date-fns	4.1.0	Date Utils	Modern date utility library for formatting and manipulation

Table 1: Complete technology stack overview

2.2 Development Tools

The project uses a modern development toolchain optimized for fast iteration and type safety. ESLint (version 9) with the Next.js configuration provides linting, while the project includes both npm and Bun lock files, allowing developers to use either package manager. TypeScript is configured with strict mode enabled, ES2017 target, and bundler module resolution, ensuring compatibility with Next.js server components. The build process uses Next.js standalone output mode, which produces a self-contained server bundle that can be deployed without the full `node_modules` directory, significantly reducing deployment footprint.

3. System Architecture

3.1 High-Level Architecture

DocsFlowBPM follows a monolithic full-stack architecture built on Next.js, where the frontend and backend coexist in a single application. The frontend is implemented as a Single Page Application (SPA) using a client-side routing approach managed by Zustand. Unlike typical Next.js applications that leverage file-based routing, DocsFlowBPM uses a single root page component (`page.tsx`) that conditionally renders different views based on the current application state. This architecture choice enables seamless transitions between views without page reloads, providing a desktop-application-like user experience. The backend exposes RESTful API routes through Next.js Route Handlers, with all business logic, authentication, and database operations executed server-side.

3.2 Frontend Architecture

The frontend architecture is built around several key patterns that ensure maintainability and performance. The central navigation state is managed by a Zustand store (`src/lib/store.ts`), which holds the current user session, authentication token, active view, cached folders, document types, and documents. The store implements a discriminated union type (`AppView`) for view routing, supporting login, dashboard, document creation/editing, admin panel, profile, and reports views. This SPA approach means all routing is handled in-memory without browser URL changes, except for supporting direct document links via query parameters.

The component hierarchy follows a clear separation of concerns. The root `page.tsx` acts as the application shell, delegating to specialized layout components based on the current view. `DashboardLayout` handles the main document management interface with sidebar, folder tree, document list/table/grid, and analytics panel. `AdminLayout` provides the administration panel with its own sidebar navigation and content area. `DocumentFormView` implements the full document editing experience with dynamic

form rendering, approval workflow integration, comments, attachments, tags, and permissions. Each major section has dedicated component files in the appropriate subdirectories under `src/components/`.

The UI is built using `shadcn/ui`, which provides over 35 pre-built accessible components based on Radix UI primitives. These include dialogs, dropdowns, command palettes, toasts, tabs, tables, tooltips, and many more. The component library uses a consistent theming system with CSS custom properties defined in `globals.css`, supporting both light and dark modes through the `next-themes` library. Custom animations are implemented in pure CSS, including fade-in, bell-shake, pulse-glow, shimmer, and glassmorphism effects.

3.3 Backend Architecture

The backend is organized as a collection of Next.js API Route Handlers under `src/app/api/`. Each domain entity has its own route file following REST conventions: documents, users, folders, document-types, tags, departments, counterparties, contacts, approvals, processes, tasks, notifications, settings, activity-log, and reports. Authentication is handled through a custom token-based session system (`src/lib/auth.ts`) using UUID tokens with 7-day expiration, stored in the Session database model. Tokens can be passed via query parameters or Authorization headers, providing flexibility for both API calls and direct browser links.

The data access layer uses Prisma ORM (`prisma/schema.prisma`) with PostgreSQL as the database. The schema defines 25 interconnected models covering all aspects of the BPM system. Prisma provides type-safe database queries, automatic migration generation, and a client singleton pattern (`src/lib/db.ts`) that prevents connection pool exhaustion during development hot reloads. The authentication middleware (`getAuthUser/extractToken`) is applied at the beginning of each API route to validate sessions and check user permissions before processing requests.

3.4 Authentication and Authorization

The system implements a role-based access control (RBAC) model with five user roles: ADMIN, DIRECTOR, CHIEF_ACCOUNTANT, ADVANCED, and USER. Authentication uses a custom session-based system with UUID tokens stored in the database. Passwords are hashed using `bcryptjs` with 12 salt rounds, and the system supports both plain text and hashed passwords during migration periods. Document-level permissions extend the RBAC model with fine-grained VIEW and EDIT access controls. When a document is created, the creator receives EDIT permission, and all privileged-role users (ADMIN, DIRECTOR, CHIEF_ACCOUNTANT) automatically receive EDIT access. Additional permissions can be granted manually through the document permissions dialog, and approval workflow steps automatically grant VIEW access to assigned approvers.

4. Database Schema

The database schema defined in `prisma/schema.prisma` consists of 25 models organized into several logical domains. PostgreSQL is used as the database provider, with all primary keys using CUIDs for globally unique identifiers. The schema supports cascading deletes for parent-child relationships, composite unique constraints for junction tables, and strategic indexes on frequently queried fields. Below is a comprehensive overview of all database models grouped by their functional domain.

4.1 Core Domain Models

Model	Description	Key Relationships
User	System users with roles, department membership, and activity tracking	Department, Session, ActivityLog, Comment, Task, etc.
Department	Organizational units for grouping users and approval assignments	User, ApprovalRouteStep, DocumentApprovalStep
Session	User session tokens with 7-day expiration	User
Document	Core entity with status, urgency, and JSON data payload	DocumentType, Folder, User, Comment, Task, etc.
DocumentType	Document type definitions with custom form schemas (JSON)	Document, DocumentTemplate, ProcessDocumentType
DocumentTemplate	Pre-configured document data templates for quick creation	DocumentType, User
Folder	Hierarchical folder structure with tree relationships	Folder (self), Document, User
DocumentTag / DocumentTagLink	Tagging system with many-to-many document-tag relationships	Document, DocumentTag, User
FavoriteDocument	User document bookmarks with unique constraints	User, Document
DocumentAttachment	File attachments with versioning, soft delete, and group tracking	Document, User
DocumentPermission	Per-document VIEW/EDIT access grants	Document, User (grantor and grantee)

Table 2: Core domain models

4.2 Approval Workflow Models

Model	Description	Key Fields
-------	-------------	------------

ApprovalRoute	Reusable approval chain templates defining step sequences	steps (embedded), approvals
ApprovalRoute Step	Individual steps in an approval route with user/department assignment	routeId, userId, departmentId, slaConfig (JSON)
ProcessDefinition	BPM process definitions with versioned step configurations	steps (JSON), status, version
ProcessDocumentType	Junction table linking processes to applicable document types	Composite PK: processId + documentTypeId
DocumentApproval	Active approval instances linked to documents	documentId, routeId, processId, status
DocumentApprovalStep	Individual steps in an active approval with decisions and SLA	stepType, conditionConfig, slaConfig, dueAt, status
ApprovalStepDecision	Immutable decision history for approval steps	decision, comment, decidedById

Table 3: Approval workflow models

4.3 Supporting Models

Model	Description
Counterparty	External organizations with INN, KPP, OGRN, and bank details
Contact	Individual contact persons linked to counterparties via junction table
Task	Approval, review, and notification tasks with priority and due dates
ActivityLog	Audit trail logging all user actions with entity references
Notification	User-facing notifications for approval requests and decisions
Comment	Document comments with user ownership and timestamps
SystemSettings	Key-value store for global configuration (working hours, upload path, etc.)

Table 4: Supporting models

5. API Routes

The backend exposes a comprehensive set of RESTful API routes organized by domain entity. All routes implement token-based authentication with consistent error handling patterns. The API supports filtering, pagination, sorting, and full-text search on documents. Below is a summary of all available API endpoints grouped by their functional domain.

Route	Methods	Description
/api/auth/login	POST	User authentication with email/password, returns session token
/api/auth/logout	POST	Session termination and token cleanup
/api/auth/me	GET	Token validation and current user profile retrieval
/api/documents	GET/POST	Document listing with filters/pagination and document creation
/api/documents/[id]	GET/PUT/DELETE	Single document CRUD with permission checks
/api/documents/search	GET	Full-text search across document titles
/api/documents/stats	GET	Dashboard analytics: status distribution, trends, top creators
/api/documents/favorites	GET/POST	User favorites management
/api/documents/recent	GET/POST	Recently viewed documents tracking
/api/documents/bulk-*	POST	Bulk operations: status change, delete, move
/api/documents/export	GET	Document data export in various formats
/api/documents/[id]/approvals	GET/POST	Approval workflow initiation and retrieval
/api/documents/[id]/attachments	GET/POST	File attachment upload and listing
/api/documents/[id]/tags	GET/POST/DELETE	Document tag management
/api/documents/[id]/permissions	GET/POST/DELETE	Document access permission management
/api/documents/[id]/comments	GET	Document comment retrieval
/api/approvals/[id]/steps/[stepId]/decide	POST	Approval step decision with SLA and condition handling
/api/folders	GET/POST	Folder tree retrieval and folder creation
/api/users	GET	User listing with search capability

/api/departments	GET/P OST	Department management
/api/document-types	GET/P OST	Document type definitions with form schemas
/api/templates	GET	Document template listing
/api/tags	GET/P OST/D ELETE	Tag management for document categorization
/api/counterparties	GET/P OST	External organization management
/api/contacts	GET/P OST	Contact person management
/api/approval-routes	GET/P OST	Approval route template management
/api/processes	GET/P OST	BPM process definition management
/api/tasks	GET	Task listing for approval workflows
/api/notifications	GET	User notification retrieval
/api/activity-log	GET	System audit trail with pagination
/api/settings	GET/P UT	Global system settings management
/api/reports	GET	Reporting data aggregation endpoints
/api/seed	GET	Database seeding for initial data population

Table 5: Complete API route listing

6. Key Features

6.1 Document Management

The document management system provides a comprehensive set of features for creating, organizing, and managing documents. Documents are created from configurable document types, each with a custom JSON form schema that supports 14 field types: text, textarea, number, money, date, email, phone, select, checkbox, switch, counterparty (directory lookup), computed (formula-based), heading, and separator. The form builder in the admin panel allows administrators to design multi-column form layouts with drag-and-drop positioning using @dnd-kit. Forms support field validation, default values, directory data sources for counterparty selection, and a powerful formula evaluator (src/lib/formula-evaluator.ts) that implements a complete expression parser with arithmetic operations, comparisons,

logical operators, and built-in functions (IF, ROUND, ABS, MIN, MAX).

Documents are organized in a hierarchical folder structure with unlimited nesting depth. Users can create, rename, and delete folders, and documents can be moved between folders using drag-and-drop. The dashboard supports both table and grid view modes, with server-side pagination (50 items per page), sorting by multiple fields, and advanced filtering by status, tags, date range, and creator. Documents can be duplicated with all their data, favorited for quick access, and bulk operations (status change, delete, move) are available for efficient management. Document numbers are auto-generated following the pattern TYPE-YYYY-NNN, ensuring unique identification within each document type and year.

6.2 Approval Workflows

The approval workflow engine is one of the most sophisticated features of the system. It supports two types of workflow definitions: Approval Routes (reusable templates) and Process Definitions (BPMN-style processes). Both define sequences of steps that documents must pass through, with each step assignable to a specific user or an entire department. The workflow engine handles three step types: APPROVAL steps requiring human decision, CONDITION steps for branching logic based on the last decision value, and STATUS_CHANGE steps that automatically update the document status when reached.

The decision-making process enforces sequential step execution, ensuring only the first pending approval step can be decided at any time. Decisions can be APPROVED, APPROVED_WITH_CHANGES, or REJECTED. When a step is rejected, all remaining steps are automatically skipped and the approval is terminated. CONDITION steps evaluate branching logic with support for both forward jumps (skipping intermediate steps) and backward jumps (resetting steps for re-approval cycles). Each approval step can have an SLA configuration (JSON) that defines maximum processing time in hours for each urgency level (LOW, MEDIUM, HIGH, CRITICAL). The SLA system (src/lib/sla.ts) calculates precise deadlines considering configurable working hours and working days, skipping weekends and non-working time slots. Decision history is immutable, stored in ApprovalStepDecision records that survive step resets, providing a complete audit trail of all approval actions.

6.3 Dashboard and Analytics

The dashboard provides a real-time overview of document activity through the stats API endpoint, which aggregates data from multiple dimensions: total document count, status distribution, documents by type, recent documents (last 24 hours), weekly and monthly totals, top creators, documents over the last 7 days, and the most active folder. The analytics panel (src/components/dashboard/dashboard-analytics.tsx) renders this data using Recharts with a donut chart for status distribution, a bar chart for time-series trends, and a horizontal bar visualization for top authors. The panel is

collapsible to save screen space and includes loading skeletons for a polished user experience during data fetching.

6.4 Additional Features

Feature	Description
Dark/Light Theme	System-wide theme switching with next-themes, persistent user preference, and OKLCH color system
Command Palette	Ctrl+K activated command menu for quick navigation, actions, and recent page access using cmdk
Notification Center	Bell icon with unread badge, notification list with types (approval requests, decisions, sharing)
Browser Notifications	Desktop notifications via Notification API when tab is in background
Document Templates	Pre-configured document data templates for quick document creation from common patterns
Keyboard Shortcuts	Comprehensive keyboard navigation with documented shortcuts dialog
Activity Log	Complete audit trail of all user actions with entity references and timestamps
File Uploads	Drag-and-drop file attachment with soft delete, versioning, and configurable storage path
Counterparty Directory	External organization management with INN/KPP/OGRN/bank details and contact linking
Auto-Save	Document auto-save with debounced timer, status indicator, and unsaved changes detection
Reports	Aggregated reporting with user activity metrics and document statistics
Export	Document data export functionality for external processing
Responsive Design	Mobile-friendly layout with sheet-based sidebar, collapsible panels, and touch-optimized controls

Table 6: Additional system features

7. Project Structure

The project follows a well-organized directory structure that separates concerns clearly between frontend components, backend API routes, shared libraries, and configuration files. The main source code resides under `src/` with `app/` for Next.js pages and API routes, `components/` for React UI components, `lib/` for shared utilities, and `hooks/` for custom React hooks. The Prisma schema and migration files are in `prisma/`, while static

assets and configuration files are at the project root level. The following table summarizes the key directories and their contents.

Path	Purpose
src/app/	Next.js App Router: pages, layouts, global CSS, and API route handlers
src/app/api/	RESTful API routes organized by domain entity (30+ endpoint files)
src/components/ui/	shadcn/ui component library (35+ accessible UI components)
src/components/dashboard/	Dashboard layout, analytics, stats bar, welcome banner
src/components/documents/	Document form, approvals, comments, attachments, permissions, tags
src/components/admin/	Admin panel: users, types, processes, tags, settings, activity log
src/components/auth/	Login page with gradient background and animated elements
src/lib/	Shared libraries: auth, db, store, types, API helper, SLA, activity log, etc.
src/hooks/	Custom React hooks: use-mobile, use-toast
prisma/	Prisma schema (25 models) and database configuration
storage/uploads/	File attachment storage organized by document ID
scripts/	Utility scripts: data import/export for database migration
public/	Static assets: logo, robots.txt, favicon
agent-ctx/	Development agent context files (work logs and task descriptions)

Table 7: Project directory structure

8. Library Modules

The `src/lib/` directory contains shared utility modules that are used across multiple components and API routes. Each module has a clear single responsibility and is designed for reusability. Below is a detailed description of each library module and its role in the system.

Module	Responsibility
auth.ts	Token-based session validation (<code>getAuthUser</code>), token extraction from requests (<code>extractToken</code>), and admin role checking (<code>isAdmin</code>)
db.ts	PrismaClient singleton with development hot-reload protection using <code>globalThis</code> pattern

store.ts	Zustand global store managing authentication state, SPA navigation (AppView union type), data cache, sidebar state, and recent pages
types.ts	TypeScript type definitions for all domain entities, form field types, document statuses, urgency levels, roles, and color mappings
api.ts	Shared API fetch helper with automatic response envelope unwrapping for consistent data extraction patterns
sla.ts	SLA deadline calculation engine with working hours support, deadline countdown formatting, and configurable urgency levels
activity-log.ts	Fire-and-forget activity logging and notification creation with browser desktop notification support
doc-permissions.ts	Document permission granting with automatic privileged-role propagation and step-based permission delegation
uploads.ts	File upload directory management with configurable base path from SystemSettings
password.ts	Password hashing (bcrypt with 12 rounds), comparison, and hash detection for migration support
formula-evaluator.ts	Complete expression parser and evaluator with tokenizer, AST builder, arithmetic/comparison/logical operations, and built-in functions
file-watchers.ts	Global file watcher registry for tracking temporary file uploads before document creation
utils.ts	CSS class name merging utility using clsx and tailwind-merge

Table 8: Library module overview

9. Deployment

The project is configured for production deployment using Bun as the JavaScript runtime. The build process (`npm run build`) uses Next.js standalone output mode, which generates a self-contained server bundle that can be deployed without the full `node_modules` directory. The build script also copies static assets and public files into the standalone directory for complete self-containment. The production server starts with `NODE_ENV=production` and Bun, outputting logs to both `stdout` and a `server.log` file for monitoring.

The Caddyfile provides a reverse proxy configuration that routes incoming requests on port 81 to the Next.js application on port 3000. It includes a special `XTransformPort` query parameter feature that allows dynamic port transformation, useful for development and testing scenarios. The Caddy configuration sets standard proxy headers (`Host`, `X-Forwarded-For`, `X-Forwarded-Proto`, `X-Real-IP`) for proper request forwarding. The system requires a PostgreSQL database with the `DATABASE_URL` environment variable, which is managed through Prisma migrations and the `db:push/db:migrate` npm scripts.

10. Document Lifecycle

Documents in DocsFlowBPM follow a well-defined lifecycle through five states, each representing a distinct phase in the document workflow. Understanding this lifecycle is essential for effective use of the system. The lifecycle begins when a user creates a document from a template or from scratch, which assigns it the DRAFT status. The document can be edited freely in this state, with auto-save ensuring no data loss. When the author is ready, they change the status to IN_PROGRESS and initiate an approval workflow.

During the approval process, the document moves through a sequence of approval steps defined by an Approval Route or Process Definition. Each step has an assigned approver (user or department) and an optional SLA deadline. The approver can APPROVE (moving to the next step), APPROVE_WITH_CHANGES (noting required modifications), or REJECT (terminating the workflow and setting the document to REJECTED status). Condition steps in the workflow can branch the approval path based on decision values, enabling complex approval logic. When all steps are completed, the document receives APPROVED or COMPLETED status depending on the process configuration. Throughout the lifecycle, all actions are logged in the ActivityLog, and relevant users receive notifications for approval requests and decisions.

Status	Description	Allowed Transitions
DRAFT	Initial creation state, free editing allowed	IN_PROGRESS, REJECTED
IN_PROGRESS	Under approval review, limited editing	APPROVED, REJECTED, DRAFT
APPROVED	All approval steps completed successfully	COMPLETED, REJECTED
REJECTED	Rejected by an approver, may require resubmission	IN_PROGRESS, DRAFT
COMPLETED	Final terminal state, no further changes	None (terminal)

Table 9: Document lifecycle states and transitions